

Harsha Raj Kumar

Nashville, TN | harsharajkumar273@gmail.com | +1 (615) 482-9742
linkedin.com/in/harsharajkumar273 | github.com/harsharajkumar-273 | Portfolio

Education

Vanderbilt University

Master of Science in Computer Science

Nashville, TN

Aug 2025 – Jun 2027

Vellore Institute of Technology (VIT)

Bachelor of Technology in Computer Science

Chennai, India

Aug 2021 – May 2025

Technical Skills

Languages: C++20 (AVX2 SIMD), C, Java, Python, Go, TypeScript, JavaScript, SQL

Systems & Infrastructure: Linux Systems Programming, `io_uring`, POSIX Threads, Docker, Git, AWS (EC2, S3, ECS), GitHub Actions

Databases & Caching: Redis (Sorted Sets), PostgreSQL (PostGIS), LevelDB, RocksDB, MongoDB

Frameworks & Web: Node.js, Express, FastAPI, React/Vite, WebSockets (Socket.io), PyTorch

CS Fundamentals: Operating Systems, Concurrent Programming, Memory Arenas, Block Bloom Filters, System Design, Data Structures & Algorithms

Experience

VU-BEAM Lab, Vanderbilt University

Systems & Software Engineer Research Assistant

Nashville, TN

Oct 2025 – Present

- Formulated physics-informed deep learning loss function metrics (ReCL) for self-supervised ultrasound representation, improving image contrast (CNR) by **85%** and outperforming CycleGAN by **20%** on clinical datasets.
- Engineered high-throughput multi-GPU PyTorch data loader pipelines to preprocess, augment, and cache **10,000+** raw high-resolution scan frames, decreasing model training latency by **40%**.
- Deployed distributed monitoring with Prometheus to log training metrics across multi-GPU nodes, preventing memory leaks and thermal throttling bottlenecks.

Vanderbilt University Mathematics Department

Platform Architect & Software Engineer

Nashville, TN

Jan – May 2024

- Engineered **Proofdesk**, a collaborative browser-based LaTeX Web IDE, reducing textbook compilation latency by **72%** (from 1.1s to **300ms**) via client-side WebAssembly (Pyodide) integration.
- Deployed WebSocket sandboxed terminal runtimes (**node-pty**) running within resource-restricted Docker containers with strict safety allocations (512MB RAM, 64 process limits).
- Architected a resilient distributed background task worker queue utilizing Redis and BullMQ, implementing local memory fallback loops to guarantee **100%** compiler availability during outages.

Projects

LSM-Tree Key-Value Engine | C++20, `io_uring`, Linux Systems, Bloom Filters

Sep – Dec 2025

- Developed a high-performance LSM-Tree database storage engine from scratch in C++20, incorporating zero-copy asynchronous WAL logging via Linux `io_uring` with `O_DIRECT` to bypass the kernel page cache.
- Designed a concurrent, lock-free SkipList MemTable utilising atomic CAS pointers, backed by a custom bump-pointer memory Arena to eliminate heap allocation fragmentation.
- Implemented 64-byte cache-aligned vectorized Block Bloom filters, restricting negative lookup search overhead to at most one CPU cache line miss.

Repost-Radar Telemetry Filter | C++20, AVX2 SIMD, MinHash, Locality-Sensitive Hashing (LSH)

Jan – Mar 2026

- Engineered a high-performance deduplication engine in C++20, accelerating signature generation by **4.2×** utilizing **AVX2 SIMD vectorization primitives** to parallelize 128 universal hash variants across CPU registers.
- Applied Locality-Sensitive Hashing (LSH) and MinHash algorithms with zero-copy token shingling (`std::string_view`) to discard redundant sensor frame payloads, reducing storage requirements by **80%**.
- Integrated concurrent slab-allocated hash indices backed by reader-writer locks (`std::shared_mutex`) to minimize lock contention under high-concurrency stream ingestion.

Production API Gateway | Node.js, Redis, SRE Circuit Breaker, EWMA PID

Jan – Mar 2026

- Designed a Node.js routing gateway supporting distributed token-bucket rate limiting via Redis Sorted Sets (`zsets`) sustaining over **25,000 requests per second** (P99 latency < 15ms).
- Built custom **SRE Circuit Breakers** with CLOSED, OPEN, and HALF-OPEN state machines to isolate faulty microservices and prevent cascading system failures.
- Integrated an adaptive rate-throttling mechanism based on an **EWMA PID controller** model, facilitating **+4%** automated throughput recovery during stress testing.

Publications & Awards

AWS Certified AI Practitioner (AIF-C01) – Amazon Web Services

July 2026

IEEE Published – An Integrated GCN–GAT–AE Framework for Robust Anomaly Detection in Industrial IoT Environments2025

Google Developer Student Club (GDSC) – Technical workshops co-lead for 200+ student members

2024 – 2025